

Linear & Logistic Regression, Regularization

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 2, Day 11, Self-study

29 September 2026

1. Where This Fits
2. Learning Outcomes
3. Inside fit: the Gradients
4. Making a Line Bend
5. The Dials: C , α , class_weight
6. Olympiad Practice
7. Check Yourself
8. Limitations and What Comes Next
9. Summary
10. References

- ▶ **In the lectures:** Day 08 introduced the line $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$, gradient descent, LinearRegression and LogisticRegression; Day 09 added MSE against MAE, log loss, Ridge and Lasso and the overfitting alarm; Day 10 showed that the line won the admissions task and lost the used-car task.
- ▶ **This deck adds** what the lectures marked “skipped on purpose”: where the gradient formulas come from, how logistic regression is trained and tuned (C, class_weight, more than two classes), how to make a line bend in scikit-learn (polynomial and interaction columns), how to read residuals, and how to let scikit-learn choose alpha and C for you.
- ▶ **How to use it:** read a frame, type its code into a Colab cell on the tables you already have (the used cars, the admissions, the ECG beats), compare with the numbers on the slide. Every number here was produced by exactly that code. Answer the three questions at the end.

Where the lecture's version breaks

- ▶ Day 10's used-car table: the forest reached MAE 15,250 and the line 20,218. Is the line simply the wrong tool, or was it fed the wrong columns? (Both: the residual plot says which columns, and a line with the right columns closes much of the gap.)
- ▶ Day 09's admissions model: accuracy 0.871, but recall only 0.722: it misses 28 of every 100 admitted students. Logistic regression has a dial for exactly that, and it does not touch the ROC-AUC.
- ▶ Day 08 said "trust the formula for the gradient". A model you cannot derive is a model you cannot debug when `ConvergenceWarning` appears.

After this deck you can

Derive the gradient of the MSE and of the log loss, explain what `LogisticRegression` does inside `fit` and what its warnings mean, tune `C` and `alpha` with the built-in CV estimators, bend a line with `PolynomialFeatures` chosen by cross-validation, and fix an imbalanced classifier with `class_weight`.

By the end of this deck, you will be able to:

- ▶ **Derive** the gradient of the mean squared error and of the log loss with the chain rule, and take one gradient step by hand.
- ▶ **Read** a residual plot and decide whether to transform the target or add a column.
- ▶ **Bend** a linear model with `PolynomialFeatures` and interaction columns, choosing the degree by cross-validation.
- ▶ **Tune** alpha and C with `RidgeCV`, `LassoCV` and `LogisticRegressionCV`, and say when the dial matters.
- ▶ **Handle** an imbalanced target with `class_weight="balanced"` and explain what it changes and what it leaves alone.
- ▶ **Apply** logistic regression to more than two classes and read its probabilities and weights.

Linear Models: **Inside fit: the Gradients**

Day 08 used $\partial L / \partial w = \frac{1}{n} \sum 2e_i x_i$ without proof. The proof, for one row and one feature, is the **chain rule**: the derivative of a function of a function is the product of the two derivatives.

$$\begin{aligned} L &= (\hat{y} - y)^2 \quad \text{with} \quad \hat{y} = w x + b && \text{loss of one row} \\ \frac{\partial L}{\partial w} &= \underbrace{\frac{\partial L}{\partial \hat{y}}}_{2(\hat{y}-y)} \cdot \underbrace{\frac{\partial \hat{y}}{\partial w}}_x = 2(\hat{y} - y)x = 2ex && \text{outer derivative times inner derivative} \\ \frac{\partial L}{\partial b} &= 2(\hat{y} - y) \cdot \frac{\partial \hat{y}}{\partial b} = 2e \cdot 1 = 2e && \text{the inner derivative is 1} \end{aligned}$$

Worked example

Row $(x, y) = (3, 7)$, $w = 0$, $b = 0$: $\hat{y} = 0$, $e = -7$, so $\partial L / \partial w = 2 \cdot (-7) \cdot 3 = -42$ and $\partial L / \partial b = -14$. Averaged over Day 08's three rows these give its -22.67 and -10 exactly.

The mean over n rows averages the per-row gradients. Week 3 applies the same chain rule layer by layer and calls it backpropagation.

Logistic regression: $z = w x + b$, $p = \sigma(z)$, loss $\ell = -[y \log p + (1 - y) \log(1 - p)]$. Three links in the chain, and one lucky simplification ($\sigma'(z) = p(1 - p)$):

$$\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial w} = \left(-\frac{y}{p} + \frac{1-y}{1-p} \right) \cdot p(1-p) \cdot x = (p - y) x$$

- So the update is $w \leftarrow w - \alpha \cdot \frac{1}{n} \sum (p_i - y_i) x_i$: the same “error times input” as for the line, with the probability p in place of $\hat{y}$ and no factor 2.

One step by hand

Three students, scaled maths score $x = (-1, 0, 2)$, labels $y = (0, 0, 1)$, start $w = 0$, $b = 0$: every $p = \sigma(0) = 0.5$.

$$\partial/\partial w = \frac{1}{3} [(0.5 - 0)(-1) + (0.5 - 0)(0) + (0.5 - 1)(2)] = \frac{1}{3} (-0.5 + 0 - 1) = -0.5,$$

$$\partial/\partial b = \frac{1}{3} (0.5 + 0.5 - 0.5) = 0.167.$$

With $\alpha = 1$: $w \leftarrow 0.5$, $b \leftarrow -0.167$. The weight moves up (the admitted student has the large x), the bias down (two of three are not admitted).

- ▶ `fit` minimises the mean log loss plus the penalty $\frac{1}{2C} \sum w_j^2$ with a **solver**: by default `lbfgs`, a cleverer cousin of gradient descent that chooses its own step size and uses the gradient above.
- ▶ It stops when the gradient is almost zero or after `max_iter` rounds (default 100). Stopping on the limit prints `ConvergenceWarning`: the weights are not at the bottom yet.
- ▶ Unscaled columns make the bowl long and narrow, so the solver zigzags and hits the limit: the warning you saw in Day 08's Lab 3 before scaling.

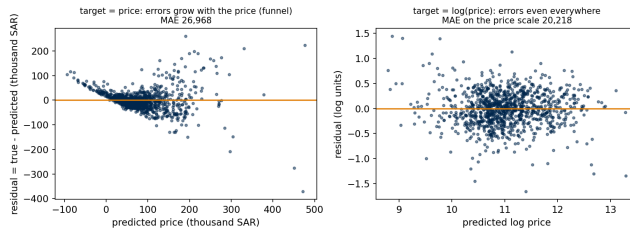
```
from sklearn.linear_model import LogisticRegression
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)) #
    scale first, then raise the limit
clf.fit(X_train, y_train)
print(clf[-1].n_iter_)           # rounds the solver actually used
print(clf[-1].coef_.shape)      # (1, 54): one weight per column, two classes
```

The common mistake

Silencing `ConvergenceWarning` with `max_iter=100000` instead of scaling. It converges eventually, but slowly, and the weights of unscaled columns stay unreadable.

Linear Models: **Making a Line Bend**

Read the Residuals Before Changing the Model



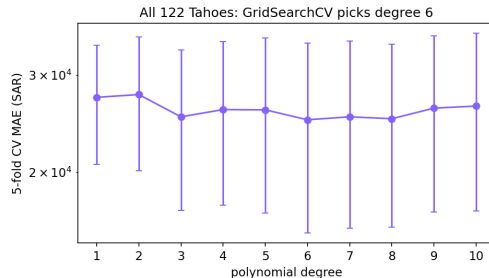
Source: Day 08's 69 used-car columns (Kaggle [turkibintalib/saudi-arabia-used-cars-dataset](#)), `LinearRegression` on the price and on its log, 20% validation split; `scripts/day11.linear_figures.py`.

- ▶ A **residual** is $y - \hat{y}$ for one validation row, plotted against the prediction; a healthy model shows a flat band of noise around zero.
- ▶ Left: a funnel, the expensive cars dominate the MSE. Right: after `np.log` on the target the band is even; same columns, MAE from 26,968 to 20,218. A curve in the band would mean a missing bent column: the next frames.

`PolynomialFeatures(degree=d)` adds the columns $x^2, x^3, \dots, x^d$ (and, with several columns, every product). The model stays linear in its weights, so fit is unchanged; only the table got wider.

```
from sklearn.preprocessing import  
    PolynomialFeatures  
pipe = make_pipeline(StandardScaler(),  
                      PolynomialFeatures(),  
                      LinearRegression())  
grid = GridSearchCV(pipe,  
    {"polynomialfeatures__degree": range(1,  
    11)},  
    cv=5, scoring="neg_mean_absolute_error")  
grid.fit(tahoe[["Mileage"]], tahoe["Price"])  
print(grid.best_params_) # degree 6
```

The dial's name inside a pipeline is
<step>__<parameter>: step names are the lower-case class names.



Source: all 122 Chevrolet Tahoe listings, mileage to price, 5-fold CV MAE with its spread; `scripts/day11.linear_figures.py`.

Degree 1: 27,348. Degrees 3 to 8 all sit near 25,000, inside each other's error bars. The grid picks 6; degree 3 is the honest choice: the simplest model inside the band.

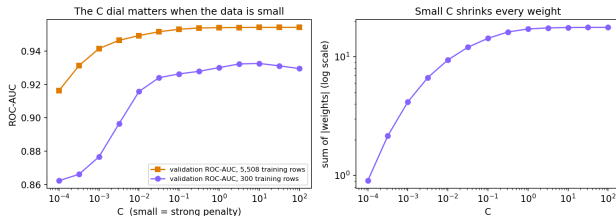
Day 10's tree found that Budapest matters only when the advanced maths exam was not taken. A line cannot say “only if”; a product column can.

```
X2 = X.copy()
X2["adv_x_budapest"] = X["math_adv"] * X["county_Budapest"]    # 1 only when
    both are 1
X2["math_x_adv"] = X["math"] * X["math_adv"]                    # score counts
    more with the exam
cross_val_score(logit, X2, y, cv=cv, scoring="roc_auc").mean()
```

admissions, logistic regression	columns	5-fold CV ROC-AUC
the 54 columns of Day 08's Lab 3	54	0.9497
plus the two hand-made products	56	0.9518
plus every square and product of 11 numeric columns (PolynomialFeatures(2), C = 0.1)	120	0.9508

- ▶ Two chosen products beat 66 blind ones; each extra column is a chance to overfit, so the penalty had to be raised. Trees and boosting find the “only if” themselves; use products with a line when a tree, a groupby or the task text names the pair.

Linear Models: **The Dials**

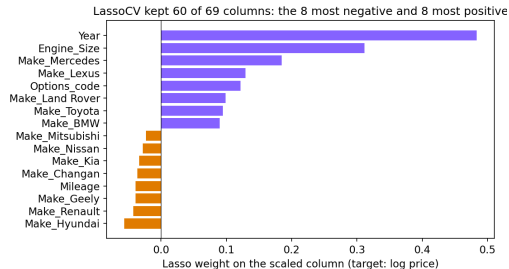


Source: HAIO 2025 admissions data, scaled, `LogisticRegression(C=...)` on 5,508 and on 300 training rows, same 1,377 validation rows; `scripts/day11.linear_figures.py`.

- ▶ $C = 1/\alpha$: **small C, strong penalty**. Below $C = 0.01$ the weights are squeezed so hard the model underfits on both tables.
- ▶ 5,508 rows: flat from $C = 0.3$ on, the data pins the weights down. 300 rows: the best C is 10 and the ends lose 0.07 of ROC-AUC. On small tables the dial is worth tuning.
- ▶ `LogisticRegressionCV(Cs=20, cv=5, scoring="roc_auc")` does the sweep inside fit: it chose $C = 0.23$ here (ROC-AUC 0.9535, same as $C = 1$).

```
from sklearn.linear_model import RidgeCV,
    LassoCV
ridge = make_pipeline(StandardScaler(),
    RidgeCV(alphas=np.logspace(-3, 3, 25)))
lasso = make_pipeline(StandardScaler(),
    LassoCV(cv=5, random_state=0, max_iter
        =20000))
ridge.fit(X_train, np.log(y_train))
lasso.fit(X_train, np.log(y_train))
print(ridge[-1].alpha_, lasso[-1].alpha_)
# 17.8    0.00127
print((lasso[-1].coef_ != 0).sum())    # 60
    of 69
```

Both try every alpha by cross-validation inside fit and keep the best. Used cars, 69 columns, log price: RidgeCV MAE 20,239, LassoCV 20,279, plain LinearRegression 20,218. On 4,228 rows the penalty neither helps nor hurts (Day 09 again).



Source: the used-car columns, LassoCV, target log price; scripts/day11.linear_figures.py.

What Lasso is for here: a **readable** model. Year +0.48, engine size +0.31, Mercedes +0.19; Hyundai and Renault below zero; nine makes at exactly 0.

class_weight="balanced": the Dial for Imbalanced Targets

Admissions: 70% not admitted. The default model minimises the average loss, so it would rather be right about the majority.

class_weight="balanced" multiplies each row's loss by $\frac{n}{2 \cdot n_{\text{class}}}$: an admitted student counts $0.5/0.298 = 1.68$ times, a rejected one $0.5/0.702 = 0.71$ times.

```
clf = make_pipeline(StandardScaler(),  
                    LogisticRegression(max_iter=2000,  
                                       class_weight="balanced"))
```

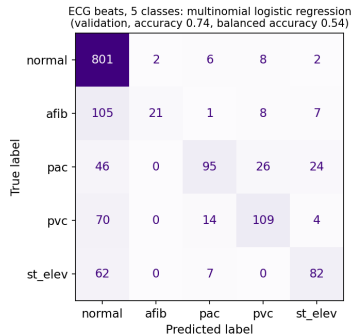
validation	default	balanced
accuracy	0.871	0.889
recall (admitted found)	0.722	0.874
balanced accuracy	0.828	0.885
ROC-AUC	0.954	0.954
share predicted "yes"	0.259	0.332

- Recall jumps by 15 points and ROC-AUC does not move: the ranking of the students is the same, the model only moved its **threshold**. Weighting is Day 09's threshold trick, built into fit.
- Use it whenever the metric is balanced accuracy, macro F1 or recall on the rare class (the Polish ECG task of Day 10 scores balanced accuracy on a 70/30 table: the organisers' write-up uses exactly this). Never for ROC-AUC alone: nothing to gain.

With k classes LogisticRegression fits **one line per class** and turns the k scores into probabilities that add to 1 (the **softmax**, Week 3's name for the k -class sigmoid). Nothing changes in your code.

```
clf = make_pipeline(StandardScaler(),  
                    LogisticRegression(  
                        max_iter=3000))  
clf.fit(X_train, y_train) # y has  
                           values 0..4  
clf[-1].coef_.shape      # (5, 150)  
clf.predict_proba(X_val)[0].round(2)  
# [0.81 0.07 0.08 0.    0.04] sums to 1
```

ECG beats (Day 10's task), 150 raw numbers per beat as columns: accuracy 0.739, balanced accuracy 0.544. The confusion matrix shows why: afib and pac are mostly called normal. A line through raw samples cannot see a wave's shape; Day 12's trees and Day 13's features can.



Source: II Olimpiada AI 2025 ECG data, 1,500 validation beats;
scripts/day11_linear_figures.py.

Linear Models: **Olympiad Practice**

Practice Task: Find the Bug in the Reference Solution

Task

Day 08's Steam prices (3,000 training games, 600 test games, MAE). A team scales the training table, fits Ridge, predicts the test table. Validation MAE 6.72, yet the submission scored worse than predicting the mean price for everyone:

```
scaler = StandardScaler().fit(F_train)
model = Ridge(alpha=1).fit(scaler.transform(F_train), y_train)
print(mean_absolute_error(y_val, model.predict(scaler.transform(F_val)))) #
6.72
test_pred = model.predict(add_features(test)) # the submission
```

- **Question 1:** what does the model receive at prediction time, and roughly what will it output? (owners is in the hundreds of thousands; a scaled column has mean 0, spread 1.)
- **Question 2:** why did validation not catch it? **Question 3:** rewrite the four lines so the bug is impossible.

Adapted from the ONIA example "Predicting Game Prices on the Steam Platform" (Olimpiada-AI, 2024/25); its advanced reference solution has this exact mismatch on

Estimated owners.

1. The test rows go in **raw**: an owners value of 350,000 lands where the model expects a number near 0. On the real table the raw-input predictions average $-633,806$ dollars and the MAE is 634,337 (correct: 6.72).
2. The validation rows *were* scaled, so validation was honest; only the last line skipped the scaler. A score from one path and a submission from another can disagree completely.
3. Put the scaler inside the model, so one object does both:

```
pipe = make_pipeline(StandardScaler(), Ridge(alpha=1)).fit(F_train, y_train)
print(mean_absolute_error(y_val, pipe.predict(F_val)))          # 6.72, same
      number
submission = pd.DataFrame({"Price": pipe.predict(add_features(test)).clip(min
      =0)})
```

Check before submitting: `submission["Price"].describe()` against `y_train.describe()`: a mean of $-633,806$ next to a training mean of 16.35 is caught in one second. Linear models, KNN and SVMs all have this trap; trees ignore scale, one more reason they make the safer first submission.

Linear Models: **Check Yourself**

Answer before turning the page.

- 1. Definition.** A classmate says “ $C=100$ means a strong penalty, so the model will underfit”. Correct the sentence, and say on which of today’s two tables (5,508 rows or 300 rows) the choice of C would matter.
- 2. Mechanism.** One student, scaled score $x = 2$, label $y = 1$, current $w = 0$, $b = 0$. Compute p , then the gradient of the log loss with respect to w and to b , then the new w and b after one step with $\alpha = 0.5$.
- 3. Code.** What is wrong with this cell, and what will the validation MAE look like?

```
scaler = StandardScaler().fit(X_train)
model = Ridge(alpha=1).fit(scaler.transform(X_train), y_train)
pred = model.predict(X_val)
print(mean_absolute_error(y_val, pred))
```

1. C is the *inverse* of the penalty: $C = 100$ is a weak penalty, so if anything the model overfits; $C = 0.001$ is the strong one. On the 5,508-row table the validation ROC-AUC was flat from $C = 0.3$ to 100; on the 300-row table it moved by 0.07, so there the dial matters.
2. $z = 0$, $p = \sigma(0) = 0.5$. Gradient for w : $(p - y)x = (0.5 - 1) \cdot 2 = -1$; for b : $(p - y) = -0.5$. Step: $w \leftarrow 0 - 0.5 \cdot (-1) = 0.5$, $b \leftarrow 0 - 0.5 \cdot (-0.5) = 0.25$. The weight rises because the admitted student has a large x .
3. The model was trained on scaled columns but `X_val` is passed raw: the scaler was never applied to the validation rows. Every input is off by its column's mean and spread, so the MAE will be enormous (on the Steam table the same bug turns an MAE of 6.8 dollars into one of thousands). Fix: `pred = model.predict(scaler.transform(X_val))`, or better, `make_pipeline(StandardScaler(), Ridge(alpha=1))` so the scaler travels with the model.

Linear Models: **Limitations & What Comes Next**

- ▶ **Bending by hand does not scale.** Two chosen products beat 66 blind ones; with 50 columns there are 1,225 pairs to choose from. Trees and boosting search that space for you (Day 12).
- ▶ **Weights are readable only after scaling, and only without strong correlation.** Two columns that move together share one effect in unpredictable halves; Lasso may keep either.
- ▶ **class_weight shifts the threshold, it does not add information.** Recall went up, ROC-AUC stayed at 0.954. If the rare class is simply not separable, no weighting helps.
- ▶ **A line through raw signal samples sees no shape.** The ECG model confused afib with normal because the wave's position and width are not linear functions of 150 voltages.

The common mistake

Scaling the training rows and forgetting the validation or test rows (today's check question). Pipelines exist so that this cannot happen.

- ▶ **Day 12, self-study:** trees and ensembles in depth, including the details beyond depth, out-of-bag scores, early stopping for boosting, and feature-engineering recipes that need no hand-picked products.
- ▶ **Day 13, self-study, Olympiad practice:** the evaluation habits that decide points: cross-validation done right, PCA, feature importance and its traps, the submission routine.
- ▶ **Week 3:** the chain rule of today's first frames, applied through many layers: neural networks and backpropagation.

Linear Models: **Summary**

- ▶ **Gradients by the chain rule:** MSE gives $2e^x$, log loss gives $(p - y)x$; both are “error times input”. LogisticRegression runs a solver on exactly this; scale first, then raise `max_iter`.
- ▶ **Residuals first:** a funnel means transform the target (log); a curve means add a bent column.
- ▶ **Bending with scikit-learn:** PolynomialFeatures in a pipeline, degree chosen by GridSearchCV; hand-chosen interaction products for “only if” effects.
- ▶ **Dials:** C is the inverse penalty and matters on small tables; RidgeCV, LassoCV, LogisticRegressionCV choose it by CV inside fit.
- ▶ **class_weight=“balanced”** raises the rare class’s recall by moving the threshold; ROC-AUC is unchanged.
- ▶ **Many classes:** one line per class, softmax probabilities that sum to 1, the same code.

Linear Models: **References & Resources**

- [1] Bin Talib, T. *Saudi Arabia Used Cars Dataset*, Kaggle [turkibintalib/saudi-arabia-used-cars-dataset](https://www.kaggle.com/datasets/turkibintalib/saudi-arabia-used-cars-dataset). Residual, degree and Lasso figures.
<https://www.kaggle.com/datasets/turkibintalib/saudi-arabia-used-cars-dataset>
- [2] Hungarian AI Olympiad (2025). *Online qualifier: University Admissions* (CC BY-NC-SA 4.0). The C-dial, class-weight and interaction experiments. <https://github.com/Hungarian-AI-Olympiad/HAIO-Hungarian-AI-Olympiad/tree/main/2025/nyari-online>
- [3] Olimpiada AI (2025). *II Olimpiada Sztucznej Inteligencji, etap 1, zadanie 3: Wykrywanie zaburzeń sygnału EKG*, data and official write-up. The multiclass figure.
https://github.com/OlimpiadaAI/II-OlimpiadaAI/tree/main/1_etap
- [4] Olimpiada-AI (2024/25). *Predicting Game Prices on the Steam Platform*, problem-proposal example with reference solutions. The practice task.
<https://github.com/Olimpiada-AI/Propunere-probleme-si-solutii>
- [5] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12. LogisticRegression, LogisticRegressionCV, RidgeCV, LassoCV, PolynomialFeatures. <https://scikit-learn.org>
- [6] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., chapter 4: training linear models, polynomial regression, regularized models, logistic and softmax regression.

[7] All plots: `slides-builder/scripts/day11_linear_figures.py` (matplotlib, real data only).